TECHNICAL DOCUMENTATION

Movie Recommendation System

Content-based, collaborative and hybrid filtering built on MovieLens 100K — the data, the algorithms, the training procedure, the evaluation, and every design decision behind them. With an interview question bank.

Author	Devesh Samant
Dataset	MovieLens `ml-latest-small` — 610 viewers, 9,724 films, 100,836 ratings
Metadata	TMDB API — 9,700 of 9,724 films enriched
Stack	Python, FastAPI, NumPy, SciPy, scikit-learn · vanilla ES modules
Repository	github.com/Deveshsamant/Movie-Recommendation-System
Licence	MIT

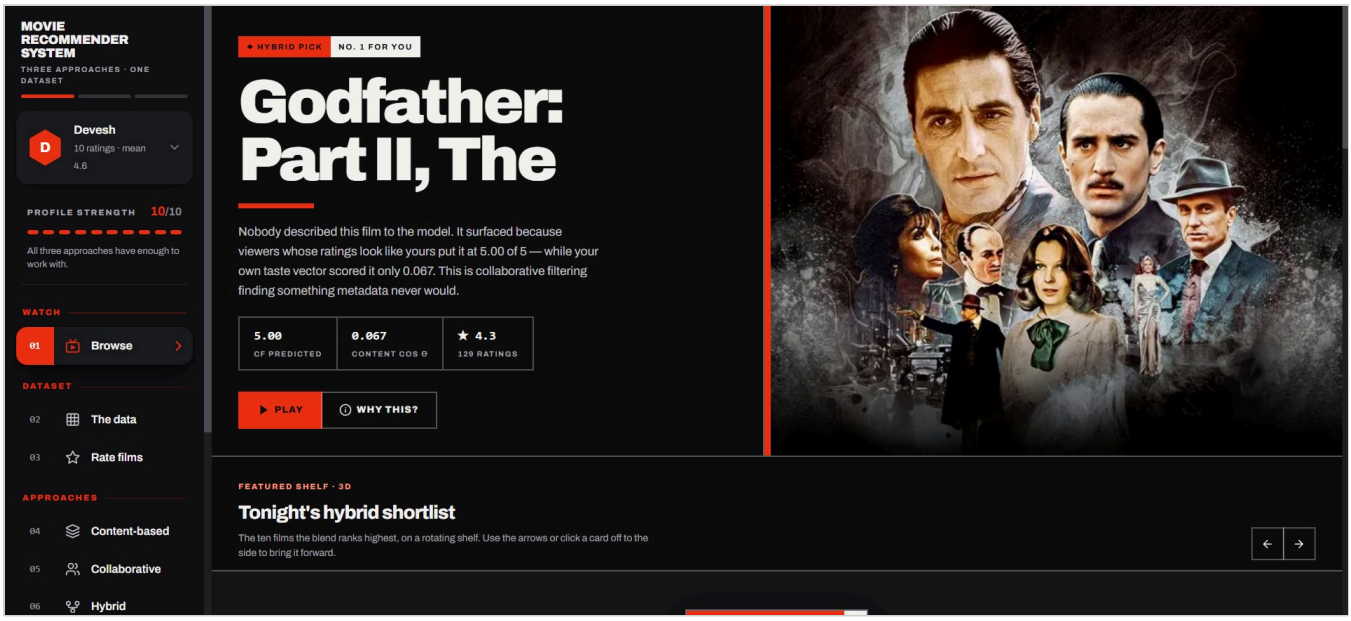
Contents

1 · The problem

1.1 Recommendation is a missing-data problem

The entire task can be stated in one sentence: **given a matrix that is almost entirely empty, predict the empty cells.**

The rating matrix here is 610 viewers by 9,724 films — **5,931,640 cells**. Only **100,836** of them contain a rating. That is **1.70%** filled. Every recommendation this system makes is an educated guess about part of the remaining **98.30%**.



The Browse view. The billboard shows the hybrid's top pick with both underlying signals exposed: collaborative filtering predicted **5.00** for this film while the content model scored it only **0.067**. The interface states which approach carried the pick rather than presenting an unexplained ranking.

1.2 The three families

APPROACH	WHAT IT READS	STRENGTH	FAILURE MODE
Content-based	Down a column — the film's own attributes	Works from the first rating; reaches the long tail	Can only return more of the same; cannot surprise
Collaborative	Across rows — who rated what	Finds genuine discoveries metadata could never suggest	A new user or new film has no row or column — *cold start*
Hybrid	Both	Covers each one's blind spot; wins on both metrics here	Inherits the cost of both; the blend weight must be tuned

> **THE CENTRAL CLAIM, MEASURED**
>
> The hybrid is not assumed to be better — it is **measured** to be better. Content + item-kNN scores **0.0904** Precision@10 against **0.0840** for item-kNN alone and **0.0228** for content alone. It beats both of its own parents on the same held-out data.

2 · The data

2.1 MovieLens — every number

The dataset is **MovieLens** `ml-latest-small`, published by GroupLens Research at the University of Minnesota. It is downloaded automatically on first run from `files.grouplens.org` (about 1 MB zipped) and is not redistributed with the source.

QUANTITY	VALUE	NOTE
Viewers (users)	**610**	each with a full rating history
Films	**9,724**	only films carrying at least one rating are kept
Ratings	**100,836**	the entire signal available
Matrix cells	5,931,640	610 × 9,724
Density	**1.70%**	98.30% unknown
Rating scale	0.5 – 5.0	half-star steps, 10 possible values
Global mean rating	3.502	used as μ in the bias model
Genres	19	plus a "no genres listed" bucket
Community tags	3,683	free-text, user-supplied
Ratings per viewer	20 / 70 / 2,698	minimum / median / maximum

The four source files are `ratings.csv`, `movies.csv`, `links.csv` (which carries the TMDB and IMDb IDs) and `tags.csv`.

2.2 The sparsity problem, quantified

Sparsity is not a footnote — it is the reason the problem is hard. Even the *densest corner* of this matrix (the 26 busiest viewers against the 40 most-rated films) is only about 86% filled, and that is a deliberately cherry-picked corner. Across the whole grid it is 1.7%.

The practical consequences:

- Most film pairs have **very few co-raters**, so a naive similarity is dominated by noise. This is what *significance shrinkage* exists to fix.
- Most users overlap on **very few films**, so user-user correlation is unstable.
- A model that recommends only popular items scores deceptively well, because popular items are the ones with data. This is why **coverage** and **novelty** are reported alongside accuracy.

2.3 The train/test split, and why it is temporal

PROPERTY	VALUE
Training ratings	**80,672** (80%)
Held-out ratings	**20,164** (20%)
Method	**Per-user temporal holdout** — the newest 20% of *each* viewer's history
Relevance threshold	held-out rating **≥ 4.0** counts as a hit
Guard	users with ≤ 6 ratings contribute no test items, so no user is left with nothing to learn from

> **WHY NOT A RANDOM SPLIT?**
>
> A random split lets the model observe a user's *future* ratings while predicting their *past*. That leaks information and inflates every metric. A temporal split asks the honest question — "given everything this person has rated so far, what will they like next?" — and is strictly harder. Expect the RMSE figures here to read slightly worse than published random-split numbers on the same dataset; they are not measuring the same thing.

Two separate sets of models are kept in memory:

- **Serving models** trained on all 100,836 ratings — used for live recommendations.
- **Evaluation models** trained on the 80% split only — used for the metrics table, so nothing is ever scored on data it was trained on.

3 · The TMDB API key and metadata pipeline

3.1 What the key is used for

The system uses **one external API: TMDB (The Movie Database)**. A single free API key is required. It is used for two distinct purposes:

1. **Presentation** — posters, backdrops, synopses, runtimes and TMDB's own rating.
2. **Modelling** — keywords, cast and directors become *features*. This is the important part: the content model has **19.6 terms per film** with TMDB enrichment and only **5.0** without it. Nearly three-quarters of the content signal comes from this API.

PROPERTY	VALUE
Provider	TMDB — `api.themoviedb.org/3`
Auth method	API key as a query parameter (`?api_key=…`)
Endpoint used	`/movie/{id}?append_to_response=credits,keywords`
Films enriched	**9,700 of 9,724** (99.75%)
Observed throughput	~2.7 films/second (latency-bound, not CPU-bound)
Storage	one JSON disk cache; each film fetched exactly once, ever

> **KEY HANDLING**
>
> The key lives in `config.json`, which is **gitignored** and never committed. A `config.example.json` ships with every hyper-parameter but a blank key. The key may alternatively be supplied through the `TMDB_API_KEY` environment variable. With no key found anywhere the application disables posters and still runs every algorithm, metric and explanation end to end.

3.2 Route failover

Several ISPs — notably a number in India — DNS-poison `api.themoviedb.org` (it resolves to a sinkhole address) while leaving the image CDN reachable. The client detects this and tries three routes in order, remembering the winner for 24 hours:

1. `api.themoviedb.org` — the normal host.
2. `api.tmdb.org` — TMDB's legacy alias, usually not on block lists.
3. The real IP address, resolved over **DNS-over-HTTPS** (Cloudflare, then Google), connected with **pinned SNI** and full certificate verification — so security is not weakened to work around the block.

3.3 Dead IDs and title-search recovery

MovieLens' `links.csv` was generated years ago. **106** of its TMDB IDs now return HTTP 404 — entries have been merged, deleted or re-issued, and several titles (*Roots*, *Generation War*, *Over the Garden Wall*) are

miniseries that were never in TMDB's `/movie` namespace at all.

When an ID fails, the client falls back to a title-and-year search across both `/search/movie` and `/search/tv`, preferring a result that actually carries artwork. This recovered **91 of the 106**. The remaining **15** receive a deliberate placeholder card rather than a blank.

4 · Feature engineering

4.1 Building the term document

Each film is converted into a "bag of terms". Field importance is expressed by **repeating a token**, which is how weighting works inside a bag-of-words model.

SOURCE	WEIGHT	PREFIX	EXAMPLE TOKEN
Genres	×3	`g_`	`g_crime`
Community tags	×2	`t_`	`t_pixar`
TMDB keywords	×2	`k_`	`k_neo_noir`
Director	×2	`d_`	`d_martin_scorsese`
Cast (top 6)	×1	`c_`	`c_robert_de_niro`
Decade	×1	`decade_`	`decade_1990s`
Title words (> 3 chars)	×1	`w_`	`w_godfather`

Prefixes keep namespaces separate — a genre called "drama" and a tag called "drama" are deliberately different features, because they carry different reliability.

4.2 TF-IDF and L2 normalisation

```
TfidfVectorizer(
    analyzer="word",
    token_pattern=r"[^ ]+",   # tokens are pre-built, do not re-split
    sublinear_tf=True,        # 1 + log(tf) — a 3rd genre repeat adds less than the 1st
    min_df=2,                 # drop terms appearing in only one film (no generalisation)
    max_df=0.6,               # drop terms in >60% of films (no discrimination)
)
```

PROPERTY	VALUE
Distinct terms in the vocabulary	**21,789**
Non-zero weights in the matrix	**190,614**
Average terms per film	**19.6**
Matrix shape	9,724 × 21,789 (sparse)
Build time	~1.6 s

WHY L2-NORMALISE?

After L2 normalisation every film vector has length 1, so `cos(a, b) = a · b` — cosine similarity collapses into a plain dot product. This is not only faster; it is what makes the explanation **exact**. Since the score is a

> sum of per-term products, each term's contribution is a real number and those numbers sum precisely to the displayed score.

5 · Algorithm 1 — Content-based filtering

Content-based filtering answers: "*what is this film about, and what else is about the same thing?*" It reads only item metadata and **never consults another user**.

5.1 The Rocchio user profile

A user is represented by a single vector in the same term space as the films. Films rated above that user's personal mean pull the profile toward them; films below push it away.

```
w_i     = (r_i − mean_u) / sd_u          # signed weight per rated film
profile = normalise( Σ_i  w_i · v_i )    # sum of weighted film vectors, then L2
```

Two details that matter:

- Dividing by the user's own standard deviation makes a generous rater and a harsh rater comparable. Without it, someone who rates everything 4–5 would have an almost meaningless profile.
- A user with zero spread (all ratings identical) would produce a zero vector, so a small positive floor (0.15) is applied — their profile becomes a plain average of what they rated.

5.2 Scoring and explanation

```
score(j) = cos(profile, v_j) = Σ_t  profile[t] × tfidf_j[t]
```

Because the profile is itself a linear combination of the films the user rated, the score decomposes **twice** — and both decompositions are exact:

By term

```
crime            0.284 × 0.311 = 0.0883
neo-noir         0.198 × 0.276 = 0.0547
Martin Scorsese  0.162 × 0.244 = 0.0395
                                --------
                          score  0.xxxx
```

By source film ("because you rated…")

```
score(j) = Σ_i  (w_i / ‖p‖) · (v_i · v_j)
```

This attributes the recommendation back to the specific films that caused it. In the running example, rating *Fargo* highly is what surfaces *The Man Who Wasn't There* and *The Big Lebowski* — the Coen brothers connection travels through the `d_joel_coen` feature.

> **THE CEILING OF THIS APPROACH**
>
> The output is a mirror of what the user already rated. That is not a bug — it is the definition. A content model has no mechanism for discovering anything outside the user's existing profile. Measured: highest

> coverage (**18.4%**, eight times any other model) and highest novelty (**6.84**), but the weakest Precision@10 (**0.0228**). Cold start, however, is a non-issue: it works from the user's very first rating.

6 · Algorithm 2 — Collaborative filtering

Collaborative filtering ignores what a film *is* entirely. Two films are similar because the same people rated them the same way. This is where genuine surprises come from — and why a brand-new film is invisible to it.

6.1 Item-based kNN

Sarwar, Karypis, Konstan & Riedl (2001).

Step 1 — mean-centre per user

Raw cosine between two film columns is dominated by the fact that some users rate everything highly. Subtracting each user's mean first removes that. The result is called **adjusted cosine similarity**.

Step 2 — significance shrinkage

```
sim'(i,j) = sim(i,j) × n_common / (n_common + λ)        λ = 25
```

A similarity of 0.9 computed from 3 co-raters is not evidence; a similarity of 0.6 from 300 co-raters is. Shrinkage scales a similarity toward zero in proportion to how little data backs it. With λ = 25, a pair with 25 co-raters keeps half its similarity.

Step 3 — keep the top k neighbours

Similarities are computed in column blocks of 512 to bound memory (a full 9,724 × 9,724 dense matrix would be 378 MB), and only the top **k = 40** per film are stored — giving a sparse matrix with **386,434** non-zero entries. Build time ≈ 10 s, then cached to disk.

Prediction

```
r̂(u,i) = mean_u + Σ_j sim(i,j)·(r_uj − mean_u) / Σ_j |sim(i,j)|
```

6.2 User-based kNN

Resnick, Iacovou, Suchak, Bergstrom & Riedl — GroupLens (1994). The original collaborative filtering algorithm. Pearson correlation between mean-centred user rows, same shrinkage, with a **fixed neighbourhood** of the k = 40 most similar users chosen once per user.

```
r̂(u,i) = mean_u + Σ_v sim(u,v)·(r_vi − mean_v) / Σ_v |sim(u,v)|
```

Because there are only 610 users, the full 610 × 610 similarity matrix fits comfortably in memory and is computed in under 0.1 s — the opposite of the item side, where 9,724 items force block processing.

6.3 Matrix factorisation — BiasSVD

Funk (2006) / Koren, Bell & Volinsky (2009). The model family that won the Netflix Prize. Implemented from scratch in NumPy rather than imported, so its internals can be exposed in the interface.

```
r̂(u,i) = μ + b_u + b_i + p_u · q_i
```

TERM	MEANING	SHAPE
`μ`	global mean rating (3.502)	scalar
`b_u`	this user's tendency to rate high or low	610
`b_i`	this film's tendency to be liked	9,724
`p_u`	the user's taste in latent-factor space	610 × 50
`q_i`	the film's position in the same space	9,724 × 50

The biases matter more than they look. They alone — with no factors at all — reach RMSE 0.9044, which is **better at ranking** than the full SVD. Everything the factors learn is the *residual* after biases are removed.

> **WHAT ARE THE LATENT FACTORS?**
>
> Nobody labels them. The factorisation invents 50 dimensions purely to compress the rating matrix, yet they land on recognisable distinctions. Inspecting factor 3 in this trained model shows *Casablanca*, *Four Weddings and a Funeral* and *Scream* at one pole against *Predator*, *Ace Ventura* and *Independence Day* at the other. The interface projects the whole 9,724 × 50 matrix to 3D with PCA so this structure can be explored directly.

6.4 The training procedure

Trained by **mini-batch stochastic gradient descent** minimising regularised squared error:

```
min  Σ (r_ui − μ − b_u − b_i − p_u·q_i)²  +  λ(‖p_u‖² + ‖q_i‖² + b_u² + b_i²)
```

Per-example gradient updates, applied with `np.add.at` so a user appearing twice in one batch receives both updates:

```
e_ui  = r_ui − r̂_ui                      # the error
b_u  += lr · (e_ui − reg · b_u)
b_i  += lr · (e_ui − reg · b_i)
p_u  += lr · (e_ui · q_i − reg · p_u)
q_i  += lr · (e_ui · p_u − reg · q_i)
```

The full loop, per epoch:

1. Shuffle all 80,672 training ratings.
2. Slice into mini-batches of 4,096.
3. Compute predictions for the batch with `np.einsum`, then the errors.
4. Scatter-add the four updates above.
5. Decay the learning rate: `lr = lr₀ / (1 + 0.03·epoch)` — so late epochs refine instead of oscillating.
6. Record training RMSE for the convergence curve.

HYPER-PARAMETER	VALUE	WHY
Latent factors	**50**	best test RMSE in the sweep; 20 underfits, 100 does not help
Epochs	**30**	past this it overfits (see below)
Learning rate	**0.007**	decayed each epoch
L2 regularisation	**0.02**	swept; 0.05 and 0.10 were both worse
Mini-batch size	4,096	tested 256–4,096; no measurable difference
Initialisation	N(0, 0.05)	seed 42, fully reproducible
Training time	~20 s	then cached to disk as `.npz`

Hyper-parameter search

A grid over `n_factors ∈ {20, 50, 100} × reg ∈ {0.02, 0.05, 0.10} × epochs ∈ {20, 30}`, scored on held-out RMSE. All 18 configurations landed between 0.889 and 0.903 — a narrow band, which itself is informative: on data this sparse the model is capacity-limited, not tuning-limited.

Detecting overfitting

Extending to 60 epochs makes the diagnosis unambiguous:

EPOCHS	TRAIN RMSE	TEST RMSE	VERDICT
30	0.677	**0.8892**	chosen
60	0.494	0.8916	train keeps falling, test rises — overfitting

6.5 Fold-in — serving a brand-new user

A user who did not exist at training time has no `p_u`. Retraining the whole model for every new signup is not viable. Instead the item factors **Q** are frozen and only the new user's parameters are solved — a small ridge regression in closed form:

```
b_u = mean( r − μ − b_i − Q_r·p )                  # bias update
p   = (Q_rᵀ Q_r + λI)⁻¹ Q_rᵀ e     where e = r − μ − b_u − b_i
```

Alternated 8 times. With 50 factors this is a 50 × 50 solve — microseconds. This is what lets the interface create a new viewer, take a handful of ratings, and immediately produce collaborative recommendations without touching the trained model.

7 · Algorithm 3 — Hybrid

The two families fail in opposite directions. Content-based cannot surprise; collaborative cannot start. A hybrid exists to let each cover the other's blind spot. Four strategies from **Burke's (2002)** taxonomy are implemented.

7.1 The four strategies

STRATEGY	MECHANISM	WHEN IT IS THE RIGHT CHOICE
Weighted	min–max normalise both signals to [0,1], then `α·CF + (1−α)·content`	Default. Simple, tunable, and the blend is inspectable.
Switching	Pick one model per user based on how much evidence exists	Cold start. Below 8 ratings it uses content and says so.
Rank fusion	Reciprocal rank fusion: `Σ 1/(k + rank)` , k = 60	When the two score scales are not comparable at all.
Cascade	Collaborative shortlists ~150 candidates, content re-ranks inside that pool	When collaborative has good recall but poor ordering.

> **THE NORMALISATION PROBLEM**
>
> A cosine similarity lives in [0, 1]; a predicted rating lives in [0.5, 5]. They cannot simply be added. Both are min–max normalised across the candidate set first, and the interface displays the raw values *and* the ranges used, so the transformation is visible rather than hidden.

7.2 Choosing alpha

α was selected by sweeping it against NDCG@10 on the held-out split, for each choice of collaborative component:

CF COMPONENT	A = 0.0	0.30	0.50	0.70	0.85	1.0
item-kNN (NDCG@10)	0.0208	0.0945	0.1150	**0.1161**	0.1127	0.1104
SVD (NDCG@10)	0.0208	0.0497	0.0525	0.0425	0.0364	0.0329

Two conclusions. First, **α = 0.70 with item-kNN** is the peak, and it beats both endpoints — which is the whole point of hybridising. Second, the choice of collaborative component matters more than α does: item-kNN is roughly twice as good as SVD at this task regardless of blend weight.

Cold-start threshold = 8 ratings. Below that, the switching hybrid falls back to content and states the reason in the interface.

8 · Evaluation

8.1 Metrics and what each answers

METRIC	QUESTION IT ANSWERS	FAMILY
RMSE / MAE	When it predicted 4.2, how close was the true rating?	Rating accuracy
Precision@10	Of the 10 films shown, how many were actually rated ≥ 4?	Ranking
Recall@10	Of everything the user went on to love, how much did the top 10 catch?	Ranking
NDCG@10	Like precision, but rewards putting good films *higher*	Ranking
MAP@10	Mean average precision — order-sensitive across the whole list	Ranking
Hit rate	Fraction of users who got at least one good film	Ranking
Coverage	Share of the 9,724-film catalogue ever recommended to anyone	Beyond-accuracy
Novelty	Mean $-\log_2$(popularity) — is it digging into the long tail?	Beyond-accuracy
Diversity	1 − mean pairwise content similarity inside one list	Beyond-accuracy

Ranking protocol: for each user, score **every** item not in their training set, take the top 10, and compare against their held-out items with rating ≥ 4. This is the strict "all unrated items" protocol — no negative sampling, which would make the numbers look far better and mean far less.

8.2 Results

MODEL	RMSE ↓	MAE ↓	PREC@10 ↑	NDCG@10 ↑	HIT RATE ↑	COVERAGE	NOVELTY
Bias baseline	0.9044	0.6995	0.0446	0.0535	0.2652	0.7%	2.38
Content-based (TF-IDF)	0.9293	0.7119	0.0228	0.0282	0.1841	**18.4%**	**6.84**
Item-based kNN	0.9376	0.7016	0.0840	0.1104	0.4139	6.0%	2.53
User-based kNN	0.9396	0.7148	0.0828	0.1081	**0.4544**	2.7%	2.05
Matrix factorisation (SVD)	0.8893	0.6830	0.0285	0.0329	0.2027	2.2%	4.12
Hybrid (content + item-kNN)	0.8933	0.6729	**0.0904**	**0.1167**	0.4307	6.5%	2.61
Hybrid (content + SVD)	**0.8683**	**0.6650**	0.0346	0.0426	0.2196	2.5%	4.04

8.3 How to read that table

1. The hybrid beats both of its own parents

0.0904 against 0.0840 (item-kNN alone) and 0.0228 (content alone). Measured on held-out data, not asserted. This is the single most important result in the project.

2. Rating accuracy and ranking quality are different problems

SVD has the best RMSE of any single model yet ranks **worse** than item-kNN — which in turn predicts ratings **worse** than the trivial bias baseline. Optimising one does not give you the other. Any project that reports only RMSE is answering a question users do not care about: nobody sees a predicted rating, they see a list.

3. Accuracy is not the only axis

The content model loses nearly every accuracy column but has **eight times** the catalogue coverage and by far the highest novelty. The bias baseline, meanwhile, recommends the same ~63 films to all 610 users (0.7% coverage) and still beats content on precision — because popularity is a genuinely strong signal and popular items are the ones with data. A production system that shipped the baseline would be accurate and useless.

9 · Engineering decisions

9.1 Ranking is not rating prediction

This is the most consequential decision in the project, and it began as a bug.

The obvious way to build a top-N list is to predict a rating for every item and sort descending. It is also wrong, for two reasons:

- **Clipping creates ties.** Predictions are clipped to [0.5, 5.0]. Hundreds of items hit exactly 5.00, and `argsort` then breaks those ties by array index — which is not a ranking at all.
- **Normalising destroys evidence.** Dividing by `Σ|sim|` is required for a calibrated rating, but it lets an item supported by *one weak neighbour* reach 5.0 and tie with an item supported by forty strong ones.

The symptom was item-kNN recommending films with 1–3 ratings in the entire dataset. The fix is to use the **un-normalised similarity-weighted sum** for ranking — the classic top-N formulation of **Deshpande & Karypis (2004)** — while keeping the normalised value for display:

```
rank(u,i)  = Σ_j sim(i,j) · (r_uj − mean_u)          # evidence strength, for ordering
r̂(u,i)     = mean_u + Σ_j sim·dev / Σ_j |sim|        # calibrated rating, for display
```

IMPACT

Item-kNN Precision@10 went from **0.002 to 0.084** — a 38× improvement from a single modelling decision, with no change to the similarity computation. Both code paths are verified to produce identical numbers where they overlap, so the arithmetic shown in the interface is always the arithmetic the ranking actually used.

9.2 Serving, caching and performance

STAGE	COLD	WARM	MECHANISM
MovieLens load	0.5 s	0.5 s	pandas
TF-IDF build	1.6 s	1.6 s	scikit-learn
Item-kNN similarity	10 s	0.03 s	sparse `.npz` on disk
SVD training	20 s	0.03 s	factors cached as `.npz`
Full evaluation	60 s	0.01 s	results cached as JSON, keyed on config
Total startup	**~2 min**	**~1.6 s**	

Every API endpoint responds in under 0.3 s. One optimisation was significant: the content explanation originally looped one sparse dot product per rated film, taking **12.2 s** for a user with 2,698 ratings. Vectorising it into a single matrix product brought that to **0.25 s** — a 49× speedup with identical output.

9.3 Notable bugs found and fixed

BUG	SYMPTOM	CAUSE
Ranking by predicted rating	Prec@10 of 0.002; films with 1 rating at the top	Clipping ties + normalisation erasing evidence strength
Content explanation 12.2 s	UI appeared to hang	Python loop instead of one matrix product
numpy.bool_ 500 error	SVD endpoint returned HTTP 500	`np.bool_` is not JSON-serialisable
Grid overflow on mobile	Horizontal page scroll	CSS grid items default to `min-width:auto`
Stale ES modules	App failed to boot after an edit	Versioning the entry point does not version its imports

10 · Limitations and future work

Limitations

- **Scale.** 610 users is tiny. Item-kNN similarity is $O(n^2)$ in items and would not survive a real catalogue without approximate nearest neighbours (LSH, FAISS, ScaNN).
- **Explicit ratings only.** Real systems are dominated by implicit feedback — clicks, watch time, skips — where an absent signal is ambiguous rather than negative.
- **No temporal dynamics.** Taste drifts. The model treats a rating from 1996 and one from 2018 identically. Koren's timeSVD++ addresses exactly this.
- **No context.** Time of day, device and companions all matter and none are modelled.
- **Offline metrics only.** Precision@10 against held-out ratings is a proxy. It cannot measure whether a recommendation *caused* a view; only an online experiment can.
- **Popularity bias.** Present and measured (via coverage and novelty) but not corrected — no inverse-propensity weighting or re-ranking for diversity.

What I would do next

1. **Implicit-feedback ALS** (Hu, Koren & Volinsky 2008) with confidence weighting — the formulation that actually matches production data.
2. **BPR** (Bayesian Personalised Ranking) — optimise the ranking objective directly instead of squared error, which is the mismatch section 9.1 works around.
3. **Two-stage architecture** — cheap candidate generation over millions of items, then an expensive re-ranker over a few hundred. This is how every production system is built.
4. **Learn the blend** instead of fixing α — a small model over both signals plus context.
5. **Online A/B testing** with interleaving, measuring long-dwell watches rather than clicks.

11 · Interview question bank

Questions an interviewer is likely to ask about this project, with answers grounded in what was actually built and measured.

A · CONCEPTUAL FOUNDATIONS

Q Explain the difference between content-based and collaborative filtering.

> Content-based reads *down a column* of the rating matrix: it describes each item by its own attributes — genre, cast, director, keywords — and recommends items whose description resembles what the user already liked. It never looks at another user.
>
> Collaborative filtering reads *across rows*: it ignores what an item *is* and asks only who rated what. Two films become similar because the same people rated them alike, even if they share no metadata whatsoever. That is precisely why it can produce surprises — and why it is helpless when an item has no ratings yet.

Q What is the cold-start problem, and which of your models suffer from it?

> Cold start is having no data to reason from. It has three forms: a new user, a new item, and a new system.
>
> All three collaborative models suffer badly — with no row or column there is nothing to correlate. Content-based does not: it works from the user's very first rating, because the item vectors exist independently of any ratings. That asymmetry is the main practical argument for hybridising. In this project the switching hybrid makes it explicit: below 8 ratings it falls back to content and displays the reason.

Q Why does a hybrid beat both of its components?

> Because their errors are not correlated. Content-based fails on items whose metadata does not capture why people like them; collaborative fails on items nobody has rated. When two models fail on different inputs, blending recovers cases either would miss alone.
>
> Measured here: content + item-kNN reaches **0.0904** Prec@10 against 0.0840 and 0.0228 for the parents. The three-way list comparison in the interface shows why — the content and collaborative top-10s frequently share **zero** films. If they agreed, blending would add nothing.

Q What is the sparsity problem?

> Only 1.7% of this matrix is filled — 100,836 ratings in 5,931,640 cells. The consequence is that most item pairs share very few co-raters, so a raw similarity is mostly noise: a correlation of 0.95 computed from three overlapping users is meaningless. It is why significance shrinkage is not optional, and why models that quietly fall back to popularity can score well.

B · THE DATA

Q Describe your dataset.

> MovieLens `ml-latest-small` from GroupLens at the University of Minnesota: **610 users**, **9,724 films**, **100,836 ratings** on a 0.5–5.0 scale in half-star steps. Density 1.7%, global mean 3.502. Each user has at least 20 ratings; the median is 70 and the heaviest user has 2,698. Metadata is enriched from TMDB, covering 9,700 of the 9,724 films.

Q Why a temporal split instead of a random one?

A random split lets the model see a user's future while predicting their past. That is leakage: knowing someone loved *The Godfather Part III* makes predicting their rating of *Part II* much easier, and the resulting numbers do not reflect deployment.

I hold out the newest 20% of *each* user's history, which mirrors the real question — given everything so far, what next? It is strictly harder, so my RMSEs read slightly worse than published random-split figures on the same data. I would rather report an honest 0.889 than a flattering 0.85.

Q How did you decide what counts as a "relevant" item?

A held-out rating of **≥ 4.0**. It is the conventional threshold on a 5-point scale and matches the intuition of "would actually recommend". I score against every item not in the user's training set rather than sampling negatives — negative sampling inflates precision substantially and makes cross-paper comparison meaningless.

C · CONTENT-BASED FILTERING

Q What is TF-IDF and why is it right here?

Term frequency × inverse document frequency. TF rewards a term appearing often in one document; IDF discounts terms appearing across many documents. The effect is that a term shared by every film — "drama" — carries little weight, while a discriminating term — "neo-noir", "Martin Scorsese" — carries a lot. That is exactly the signal a similarity should be built on.

I use **sublinear TF** (1 + log tf) so that the third repetition of a genre adds less than the first, and `min_df=2` / `max_df=0.6` to drop terms that are too rare to generalise or too common to discriminate.

Q Why L2-normalise the vectors?

Two reasons. Practically, once every vector has unit length, cosine similarity *is* the dot product — cheaper and simpler.

More importantly it makes the explanation exact. The score becomes a sum of per-term products, so each term's contribution is a real number and those numbers sum precisely to the displayed score. The interface can show "crime contributed 0.0883" and that is arithmetic, not an approximation.

Q How do you build a user profile from ratings?

A **Rocchio** profile — a weighted sum of the item vectors the user rated, where the weight is `(r - mean_u) / sd_u`. Items rated above the user's own mean pull the profile toward them; items below push it away. Dividing by their standard deviation makes a generous and a harsh rater comparable.

Because the profile is a linear combination, the final score decomposes back across the source films — which is how the interface says "because you rated *Fargo* 5.0".

Q How do you weight different metadata fields?

By repeating tokens before vectorisation — that is how weighting works in a bag-of-words model. Genres ×3, tags ×2, TMDB keywords ×2, director ×2, cast ×1. Each field also gets a namespace prefix so a genre "drama" and a tag "drama" stay distinct features, since they differ in reliability.

The weights are hand-set from judgement about signal quality, not learned. Learning them against NDCG would be a genuine improvement.

D · COLLABORATIVE FILTERING

Q What is adjusted cosine similarity and why mean-centre?

Plain cosine between two item columns is dominated by overall rating generosity: two films look similar simply because the same lenient users rated both highly. Subtracting each *user's* mean before computing the cosine removes that user-level offset, so the similarity measures agreement about *relative* preference — did the same people like this film more than they usually like things?

Q What is significance shrinkage and why do you need it?

`sim' = sim × n / (n + λ)` with λ = 25. It scales a similarity toward zero in proportion to how few co-raters support it.

Without it, a pair of obscure films rated by the same two people can show similarity 1.0 and dominate the neighbourhood. With λ = 25, a pair with 25 co-raters keeps half its similarity, and one with 3 keeps about a tenth. On a matrix this sparse it is the difference between a working model and noise.

Q Item-based or user-based kNN — when would you choose each?

Item-based is usually preferred in production: item-item similarities are far more stable than user-user (an item's rating profile changes slowly, a user's grows constantly), and they can be precomputed offline. It also explains well — "because you liked X".

User-based is attractive when users greatly outnumber items, or when you want social framing ("people like you also enjoyed"). Here they perform almost identically (Prec@10 0.0840 vs 0.0828), but item-kNN has better coverage — 6.0% against 2.7%.

Q Explain matrix factorisation. What do the latent factors represent?

Approximate the sparse rating matrix R as the product of two thin matrices: users × k and k × items. Each user and each item becomes a vector of k latent factors, and a prediction is their dot product — how well the user's taste aligns with what the film offers.

The factors are not labelled by anyone; they are whatever directions best compress the observed ratings. They frequently land on interpretable axes. In my trained model, factor 3 separates *Casablanca* and *Four Weddings and a Funeral* at one pole from *Predator* and *Ace Ventura* at the other. The interface projects the full 9,724 × 50 matrix to 3D by PCA so this can be explored directly.

Q Why include bias terms? What do they do?

> `r̂ = μ + b_u + b_i + p_u·q_i`. The biases capture effects that have nothing to do with taste matching: some users rate generously, some films are broadly liked. If you leave them out, the factors waste capacity encoding them.
>
> They explain a surprising amount. Biases alone reach RMSE 0.9044 — and out-*rank* the full SVD. Everything the factors learn is the residual after biases are removed.

Q How do you serve a user who signed up one minute ago?

> **Fold-in.** Freeze the item factors Q and solve only for the new user's `p_u` and `b_u`. With Q fixed the objective is ridge regression with a closed form:
>
> ```
> p = (Q_rᵀ Q_r + λI)⁻¹ Q_rᵀ e
> ```
>
> alternated a few times with the bias update. At 50 factors that is a 50 × 50 solve — microseconds, no retraining. It is what lets the demo create a viewer, take five ratings and immediately produce collaborative recommendations.

E · TRAINING

Q Walk me through your SGD training loop.

> Per epoch: shuffle all 80,672 training ratings, slice into mini-batches of 4,096. For each batch, compute predictions with `np.einsum`, take the errors, then apply four scatter-add updates — to `b_u`, `b_i`, `p_u` and `q_i` — each of the form `param += lr · (error · other − reg · param)`. I use `np.add.at` so a user appearing twice in a batch gets both updates rather than one silently overwriting the other. The learning rate decays as `lr₀/(1 + 0.03·epoch)` so late epochs refine instead of oscillating. 30 epochs, about 20 seconds, then cached.

Q How did you choose 50 factors and 30 epochs?

> Grid search on held-out RMSE over factors {20, 50, 100} × reg {0.02, 0.05, 0.10} × epochs {20, 30}. 50 factors with reg 0.02 at 30 epochs won at 0.8892.
>
> Worth noting: every one of the 18 configurations landed between 0.889 and 0.903. That narrow band is itself a finding — on data this sparse the model is capacity-limited, not tuning-limited, and chasing hyper-parameters has a low ceiling.

Q How did you know it was overfitting?

> By watching train and test RMSE diverge. At 30 epochs: train 0.677, test 0.8892. Extending to 60: train falls to 0.494 while test *rises* to 0.8916. Training error improving while held-out error worsens is the definition. I stopped at 30.

Q What is your regularisation doing?

> L2 on every learned parameter — both factor matrices and both bias vectors — appearing in the update as the `− reg · param` term, which pulls parameters toward zero every step. It matters most for items with few ratings, whose factors would otherwise fit noise. λ = 0.02 was swept; 0.05 and 0.10 both scored worse, suggesting the model is already capacity-constrained.

F · EVALUATION

Q Your SVD has the best RMSE but ranks worse than kNN. Explain that.

They are different objectives. RMSE measures calibration across *all* predictions; ranking cares only about the ordering of the *top few*. Squared error spends capacity being accurate about films the user will never see.

Concretely: SVD is excellent at knowing an average film is a 3.4, which lowers RMSE a lot and affects the top 10 not at all. Item-kNN is poorly calibrated overall — worse than the bias baseline — but very good at identifying which handful of items a user's neighbourhood loved. That is why the project reports both, and why the two hybrids win different columns.

Q Precision@10 of 0.09 sounds terrible. Is it?

It is in line for this protocol. Three things suppress it. First, I rank against **all** ~9,700 unrated items with no negative sampling. Second, the split is temporal, which is harder than random. Third, the ceiling is low by construction: the median user has only about 14 held-out items, of which perhaps 8 are relevant — so even a perfect top-10 could not exceed roughly 0.8, and the realistic ceiling is far below that.

What matters is the *relative* comparison on identical data: 0.0904 versus 0.0840 versus 0.0228 is a real ordering. An absolute Prec@10 is only meaningful alongside its protocol.

Q What is NDCG and why use it over precision?

Normalised Discounted Cumulative Gain. Each hit contributes gain discounted by $1/\log_2(\text{rank} + 1)$, and the total is normalised by the best achievable ordering. Precision@10 treats a hit at position 1 and position 10 identically; NDCG rewards putting good items higher — which is what users experience, since attention decays sharply down a list.

Q Why measure coverage and novelty at all?

Because accuracy alone can hide a useless system. My bias baseline recommends the same ~63 films to all 610 users — **0.7% coverage** — and still beats the content model on precision. It is accurate and worthless: no personalisation, no discovery, and no way for the long tail of the catalogue ever to be seen.

Coverage measures how much of the catalogue is reachable; novelty (mean $-\log_2$ popularity) measures how far from the blockbusters the recommendations sit. The content model wins both by a wide margin, which is a real strength the accuracy columns completely miss.

G · ENGINEERING

Q What was the hardest bug you found?

Ranking top-N by predicted rating. Item-kNN was scoring **Prec@10 of 0.002** — essentially random — and recommending films with one or two ratings in the entire dataset.

Two causes. Clipping predictions to 5.0 produced hundreds of exact ties, resolved by array index. And dividing by $\Sigma|sim|$ — necessary for a calibrated rating — let an item supported by one weak neighbour tie with one supported by forty strong ones.

The fix was to separate the two jobs: rank on the un-normalised weighted sum (Deshpande & Karypis's top-N formulation), display the normalised rating. Prec@10 went from **0.002 to 0.084**, a 38× improvement, without touching the similarity computation.

Q How do you keep the explanation honest?

Deliberately, because it is easy to fake. Every model exposes both a `rank_all` and a `score_all` path, and I verified they produce *identical* numbers where they overlap — so the arithmetic displayed is always the arithmetic the ranking used, never a plausible-looking reconstruction. The content explanation is exact by construction, since cosine of normalised vectors is a dot product and the per-term contributions sum to the score.

Q How would you scale this to 10 million users and a million items?

The kNN similarity is $O(n^2)$ in items — 10^{12} pairs is not viable. I would move to a **two-stage architecture**, which is how production systems are actually built:

1. **Candidate generation** — a few hundred items from millions, cheaply: approximate nearest neighbours over learned embeddings (FAISS/ScaNN), plus popularity and recency sources.
2. **Ranking** — an expensive model over only those few hundred, with full features.

Factorisation would move to ALS or BPR on Spark, retrained nightly, with fold-in for new users between runs. Embeddings go in a vector store, item-item neighbours precomputed offline into a key-value store.

Q How do you make it fast enough to serve?

Everything expensive is precomputed and cached: item-kNN similarities as a sparse matrix, SVD factors as arrays, evaluation results as JSON — all keyed on the data and hyper-parameters so they invalidate correctly. Cold start is about two minutes; warm start 1.6 seconds. Serving a request is then a sparse matrix–vector product.

The one place I got it wrong was the content explanation: it originally looped one dot product per rated film, which took 12.2 s for the heaviest user. Vectorising into a single matrix product made it 0.25 s.

H · HARDER FOLLOW-UPS

Q Your dataset has 610 users. Is anything you learned actually valid?

The *relative* comparisons are valid — every model saw identical data and an identical protocol, so the ordering between them is real. The *absolute* numbers are not transferable; Prec@10 of 0.09 says little about a system with a million users.

What generalises is the methodology: temporal splitting, separating ranking from rating prediction, reporting beyond-accuracy metrics, and shrinking similarities by support. What does not generalise is the specific tuning. I would treat every hyper-parameter here as a starting point, not a conclusion.

Q How would you handle implicit feedback instead of ratings?

The key difference is that absence is ambiguous — not watching something might mean dislike, or might mean never seeing it. So you cannot simply treat unobserved as zero.

I would use the **Hu, Koren & Volinsky (2008)** formulation: model a binary preference plus a *confidence* that scales with observation count, and train over all pairs with ALS, which has an efficient closed-form alternation. Or BPR, which optimises pairwise ordering directly. Evaluation would also change — ranking metrics only, since there is no rating to be accurate about.

Q How would you A/B test this?

> Randomise by user, not by session, so a person gets a consistent experience. Primary metric should be something that reflects satisfaction rather than curiosity — long-dwell watches or completion rate, not click-through, which rewards misleading thumbnails. Guardrail metrics: catalogue coverage and long-tail share, so a win is not purely popularity bias.
>
> For faster signal I would use **interleaving** — mixing both rankers' results in one list and attributing clicks — which needs far less traffic than a split test. And I would run long enough to see novelty effects decay.

Q What would you do differently if you started again?

> Three things. I would separate ranking from rating prediction *from the start* rather than discovering it as a 38× bug. I would build the evaluation harness before the models, so every change was measured immediately instead of retrofitted. And I would learn the metadata field weights against NDCG rather than hand-setting genres ×3, tags ×2 — those were judgement calls that were never validated.

Q Why did you build the explanations at all? Users do not read maths.

> Most will not, and in a consumer product the exposed arithmetic would be a toggle that stays off — which is exactly how it is built here.
>
> But the explanations were not primarily for users; they were for me. Being unable to show *why* an item ranked where it did is how the ranking bug survived — the list looked plausible while Precision@10 was 0.002. Once every score had to decompose into its parts, an item topping the list on the strength of one weak neighbour became obvious. Explainability was a debugging tool before it was a feature, and it is also increasingly a regulatory requirement.